> for both the online and offline version does it make sense to use inspect-ai and if so how

Yes — **Inspect AI is actually a very good fit for both the offline and online versions**, but it should be used as a **testing + orchestration layer**, not as the entire benchmark system.

Think of it this way:

> Inspect AI = experiment runner + evaluation harness
> Your system = data platform + ground truth + scoring pipeline

They complement each other rather than replace each other.

---

1. Where Inspect AI fits best

Inspect AI (from UK AI Safety Institute / UK AISI ecosystem) is strongest at:

- Running LLMs as "agents" over structured inputs
- Reproducible evaluation runs
- Prompt/version control
- Scoring outputs
- Parallel execution across models
- Structured datasets (tasks → outputs → scorers)

So it maps very cleanly to your **review agent evaluation layer**.

---

2. How it fits into your architecture

Offline benchmark

```
GitLab Snapshots
      ↓
Inspect Dataset Loader
      ↓
Inspect Tasks (one per PR)
      ↓
GPT-5 / Claude / Hybrid agents
      ↓
Inspect Scorers (LLM judge + rules)
      ↓
Metrics (precision/recall)
```

Online benchmark

```
GitLab Webhook PR
      ↓
Snapshot Builder
      ↓
Inspect Task Queue
      ↓
Agents run via Inspect
      ↓
Store findings
      ↓
Later: ground truth matching system (your system)
```

Key distinction:

- Inspect runs the **model evaluation**
- Your system owns **ground truth + long-term outcomes**

---

3. What Inspect replaces (and what it doesn't)

✔️ Inspect can replace

1. Agent execution layer

Instead of writing your own runner:

```
Python
```

```
run_gpt5(diff)
run_claude(diff)
run_hybrid(diff)
```

You define Inspect tasks:

Python

Run

```
Task: "review_pr"
Input: diff + metadata
Models: GPT-5, Claude, Hybrid
Output: structured findings
```

---

2. Parallel execution

Inspect gives you:

- batching
- concurrency
- retries
- logging
- reproducibility

This is a big win.

---

3. Basic scoring

For offline benchmarks, Inspect can compute:

- exact match scoring
- LLM-as-judge scoring
- rubric-based scoring

❌Inspect should NOT replace

1. Ground truth construction

You still need your system for:

- commit → bug linkage

- incident mapping
- review comment normalization
- clustering issues

Inspect is not designed for this.

---

2. Long-term outcome tracking (online benchmark)

Inspect does NOT handle:

- 7–30 day delayed evaluation
- production incidents
- hotfix tracking
- PR follow-up evolution

That stays in your system.

---

3. GitLab/GitHub ingestion layer

Inspect is not a data collector.

You still need:

- GitLab MR collector (you already built this)
- snapshot system
- normalization layer

---

4. How to structure Inspect for your system

4.1 Core abstraction

Each PR becomes an Inspect “task”:

```
Task = {
  input: PR snapshot,
  target: model-generated findings,
  scorer: LLM judge + rules
}
```

4.2 Example Inspect task (conceptual)

Python

Run

```python
from inspect_ai import task, eval

@task
def review_pr():
    return {
        "input": {
            "diff": "...",
            "files": [...],
            "metadata": {...}
        },
        "solver": [
            "gpt5",
            "claude",
            "hybrid"
        ],
        "scorer": "llm_judge"
    }
```

4.3 Scorer integration (VERY important for you)

You plug in your **existing LLM judge system**:

```
Inspect scorer
      ↓
Your Judge #1 (semantic match)
Your Judge #2 (code-aware match)
Ensemble decision
```

So Inspect becomes the execution engine, not the intelligence layer.

5. Offline benchmark with Inspect (recommended design)

Pipeline

```
GitLab snapshots
      ↓
Inspect dataset loader
      ↓
Each PR = task
      ↓
Run agents:
   – GPT-5
   – Claude
   – Hybrid
      ↓
Your LLM matcher system
      ↓
Metrics aggregation
```

What Inspect gives you here

Strong benefits:

- reproducible runs across models
- consistent prompts
- clean experiment tracking
- easy ablation tests:
  - GPT-5 vs Claude
  - GPT-5 vs Hybrid
  - prompt variations
- parallel evaluation across 1000+ PRs

What you still own

- ground truth graph
- bug clustering
- severity weighting
- long-term outcome linking

---

6. Online benchmark with Inspect (recommended hybrid

use)

This is where people often misuse Inspect — don't.

Correct usage pattern

Step 1 (your system)

```
GitLab webhook → snapshot → queue
```

Step 2 (Inspect)

```
Run review agents in Inspect
```

Step 3 (your system)

```
Store findings → wait 7–30 days → evaluate outcomes
```

Why this separation matters

Inspect is stateless.

Your online benchmark is **stateful over time**.

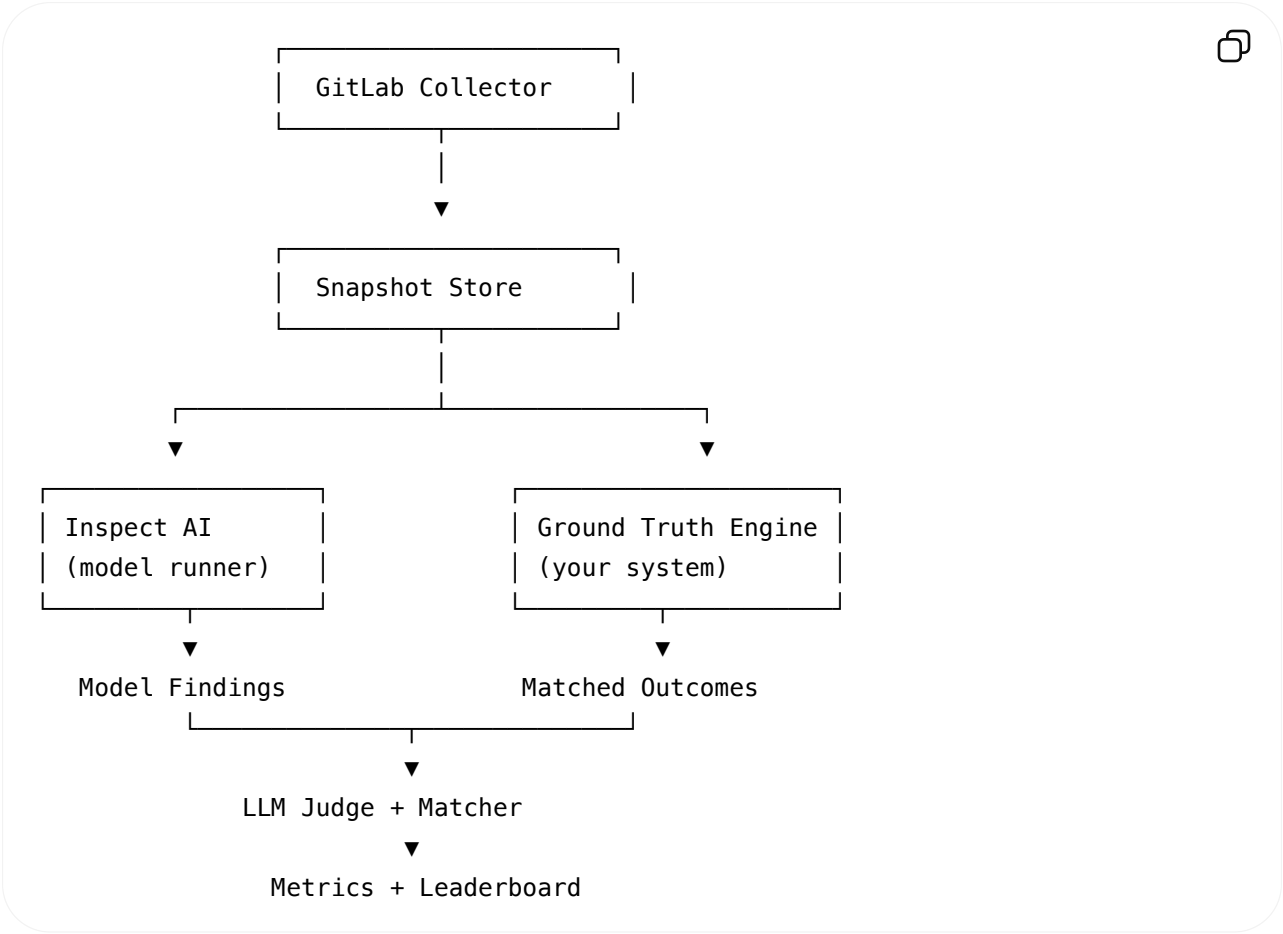
So:

- Inspect = "compute step"
- Your system = "temporal reasoning system"

---

7. Best-practice architecture (what I recommend)

Final system split



---

8. When Inspect AI is NOT worth it

You might skip Inspect if:

- you want full control over orchestration
- you need deep GitLab-native workflows
- you already have a distributed job system (Airflow, Temporal, Dagster)
- you want tight coupling with your LLM judge system

In that case, Inspect becomes optional.

---

9. When Inspect AI is highly valuable

Use it if you want:

- fast experimentation across GPT-5 / Claude / hybrid
- clean benchmark reproducibility
- standardized evaluation harness

- minimal infra for running models at scale

---

10. Simple rule of thumb

> Use Inspect AI for “running experiments”
> Use your system for “defining truth”

If you want, the next step I can design is:

👉A concrete Inspect + GitLab + LLM Judge integration repo layout

or

👉A full production-grade “benchmark execution engine” (queues, workers, retries, caching, cost control)

Vanta · Sponsored

Get ready for AI audits
Turn AI oversight into a practical workflow.

A concrete Inspect + GitLab + LLM Judge integration repo layout

Here’s a **concrete, production-oriented repo layout** that cleanly integrates:

- GitLab collector (your data plane)
- Inspect AI (model execution + experiment runner)
- LLM Judge system (your semantic ground truth matcher)
- Offline + online benchmark modes
- Scoring + leaderboard

The key design principle is separation of concerns:

> Inspect runs models. Your system defines truth. A scoring layer connects them.

🧰 Repository Layout

```
code-review-benchmark/
│
├── README.md
├── pyproject.toml
├── docker-compose.yml
├── .env.example
│
├── config/
│   ├── inspect.yaml
│   ├── models.yaml
│   ├── gitlab.yaml
│   └── scoring.yaml
│
├── data/
│   ├── raw/                  # GitLab snapshots (immutable)
│   ├── processed/           # normalized PRs
│   ├── ground_truth/        # clustered issues
│   └── embeddings/         # cached vectors
│
├── collectors/
│   └── gitlab/
│       ├── client.py        # GitLab API wrapper
│       ├── fetch_mrs.py     # MR discovery
│       ├── snapshot.py      # full MR snapshot builder
│       └── sync.py          # incremental sync jobs
│
├── inspect_runs/
│   ├── tasks/
│   │   ├── review_pr.py     # main Inspect task definition
│   │   ├── prompts.py       # model prompts
│   │   └── schemas.py       # structured outputs
│   │
│   ├── solvers/
│   │   ├── gpt5.py
│   │   ├── claude.py
│   │   └── hybrid.py
│   │
│   ├── scorers/
│   │   ├── llm_judge.py     # core semantic judge
│   │   ├── rule_scorer.py   # deterministic checks
│   │   └── ensemble.py      # final decision
│   │
│   └── runner.py            # executes Inspect jobs
│
├── judge/
│   ├── semantic_matcher.py   # Judge #1 (meaning equivalence)
│   └── code_validator.py     # Judge #2 (diff-aware validation)
```

```
│   │   code_validator.py     # Judge #2 (diff-aware validation)
│   ├── severity.py           # severity alignment scoring
│   └── cluster.py            # ground truth clustering
│
├── benchmark/
│   ├── offline/
│   │   ├── build_dataset.py
│   │   ├── run_offline.py
│   │   └── evaluate.py
│   │
│   ├── online/
│   │   ├── webhook_handler.py
│   │   ├── queue.py
│   │   ├── scheduler.py
│   │   └── evaluator.py
│   │
│   └── metrics/
│       ├── precision.py
│       ├── recall.py
│       ├── severity_weighted.py
│       └── leaderboard.py
│
├── pipelines/
│   ├── offline_pipeline.py
│   ├── online_pipeline.py
│   └── nightly_recompute.py
│
├── storage/
│   ├── db.py                # Postgres interface
│   ├── models.py            # ORM models
│   └── migrations/
│
├── api/
│   ├── server.py            # FastAPI service
│   ├── routes/
│   │   ├── benchmarks.py
│   │   ├── runs.py
│   │   └── leaderboard.py
│   └── schemas/
│
├── workers/
│   ├── inspect_worker.py    # runs Inspect jobs
│   ├── judge_worker.py     # runs LLM matching
│   └── sync_worker.py      # GitLab sync daemon
│
├── dashboard/
│   ├── streamlit_app.py
│   └── components/
│
```

```
└── tests/
    ├── test_collector.py
    ├── test_judge.py
    ├── test_matching.py
    └── test_pipeline.py
```

🔄End-to-End Data Flow

1. GitLab ingestion

```
GitLab MR
   ↓
collectors/gitlab/snapshot.py
   ↓
data/raw/{project}/{mr}.json
```

---

2. Inspect execution layer

```
raw snapshot
   ↓
inspect_runs/tasks/review_pr.py
   ↓
GPT-5 / Claude / Hybrid
   ↓
structured findings
```

Stored:

```
data/processed/findings.json
```

---

3. LLM Judge system

```
findings + ground truth
   ↓
judge/semantic_matcher.py
judge/code_validator.py
   ↓
match graph
```

Stored:

```
data/ground_truth/match_graph.json
```

---

4. Scoring layer

```
match graph
   ↓
metrics/
   ↓
precision / recall / severity score
   ↓
leaderboard
```

🧠Inspect Integration (Core File)

`inspect_runs/tasks/review_pr.py`

Python Run

```python
from inspect_ai import task

@task
def review_pr(snapshot):
    return {
        "input": {
            "diff": snapshot["diff"],
            "files": snapshot["files"],
            "metadata": snapshot["metadata"],
        },
        "solvers": ["gpt5", "claude", "hybrid"],
        "scorer": "llm_judge"
    }
```

🤖Solver Layer (Models)

inspect_runs/solvers/gpt5.py

Python Run

```python
def run_gpt5(prompt: str) -> str:
    # wrapper around API
    return call_gpt5(prompt)
```

---

inspect_runs/solvers/hybrid.py

```
PR
 ├─ Semgrep
 ├─ CodeQL
 └─ GPT-5 synthesis
```

⚖️LLM Judge (Critical Component)

judge/semantic_matcher.py

Python Run

```python
def semantic_match(agent_finding, ground_truth):
    prompt = f"""
    Do these describe the same underlying bug?

    Agent:
    {agent_finding}

    Ground Truth:
    {ground_truth}

    Answer MATCH or NO_MATCH with confidence.
    """
    return call_llm(prompt)
```

---

judge/code_validator.py

Python Run

```python
def validate_against_diff(finding, diff):
    prompt = f"""
    Given this code diff:

    {diff}

    Does the finding correspond to a real defect in the code change?
    """
    return call_llm(prompt)
```

---

judge/ensemble.py

Python Run

```python
def final_decision(j1, j2):
    if j1["match"] and j2["match"] and j1["confidence"] > 0.7:
        return True
    return False
```

Metrics Layer

benchmark/metrics/precision.py

Python

```python
def precision(tp, fp):
    return tp / (tp + fp + 1e-9)
```

---

benchmark/metrics/recall.py

Python

```python
def recall(tp, fn):
    return tp / (tp + fn + 1e-9)
```

🔁 Offline Pipeline

pipelines/offline_pipeline.py

Python

```python
def run_offline():
    snapshots = load_snapshots()

    for pr in snapshots:
        findings = run_inspect(pr)
        ground_truth = load_ground_truth(pr)

        matches = run_judge(findings, ground_truth)
        score = compute_metrics(matches)

    save_leaderboard(score)
```

🌐 Online Pipeline

pipelines/online_pipeline.py

Python

Run

```python
def handle_new_mr(event):
    snapshot = build_snapshot(event)

    findings = run_inspect(snapshot)

    store_findings(findings)

    # WAIT 7–30 days before evaluation
```

---

benchmark/online/evaluator.py

Python

Run

```python
def evaluate_after_delay(mr_id):
    findings = load_findings(mr_id)
    ground_truth = build_ground_truth(mr_id)

    matches = run_judge(findings, ground_truth)
    update_metrics(matches)
```

⚙️Workers

Inspect worker

```
consumes queue → runs Inspect → stores outputs
```

Judge worker

```
consumes findings → runs LLM matcher → stores graph
```

Sync worker

```
GitLab → snapshot → DB
```

📈Dashboard

- per-model precision/recall
- severity-weighted recall
- cost per PR
- false positive heatmaps
- repo-level breakdown

🧠Key Design Insight

This architecture cleanly separates:

Layer	Responsibility
GitLab Collector	Data ingestion
Inspect AI	Model execution
Judge System	Semantic truth mapping
Metrics Engine	Evaluation
Online Pipeline	Temporal evaluation

🚀Minimal MVP Path

If you want to build this incrementally:

Phase 1

- GitLab collector
- Inspect runner (GPT-5 + Claude)
- JSON output

Phase 2

- LLM judge (single model)
- basic precision/recall

Phase 3

- hybrid agent
- embedding filtering

Phase 4

- online delayed evaluation
- leaderboard UI